Lab 5: Static Code Analysis

Course: Version Control & Software Engineering (VCS) | **Student ID:** PES1UG25AM818
Assigned Repository: L5_65_auction-bidding-main | **File:** auction_system.py

---

1. Executive Summary

This laboratory exercise performed static code analysis, vulnerability remediation, and quality refactoring on an online auction and bidding system (auction_system.py). Using industry-standard tools (**Bandit**, **Flake8**, and **Pylint**), 3 security vulnerabilities, 19 PEP 8 style violations, and critical business logic bugs (lexicographical date sorting, shill bidding, and missing reserve price enforcement) were identified and resolved across three scoped Git commits, elevating Pylint score to a perfect **10.00/10**.

2. Static Analysis Metrics (Before vs After)

Tool	Focus Area	Before Refactoring	After Refactoring	Status
Bandit	Application Security (AST Scan)	3 Issues (2 High, 1 Med)	**0 Issues**	**PASS**
Flake8	PEP 8 Style, Whitespace, Length	19 Violations	**0 Violations**	**PASS**
Pylint	Code Quality, Design, Docstrings	6.09 / 10.00	**10.00 / 10.00**	**PASS**

3. Identified Issues & Remediation Log

Issue / Code	Tool	Sev	Description	Fix Approach
B411: xmlrpc	Bandit	High	Vulnerable XML-RPC client	Applied defusedxml.xmlrpc.monkey_patch()
B701: autoescape	Bandit	High	Jinja2 autoescape=False (XSS)	Enabled autoescape=True in Environment
B704: Markup XSS	Bandit	Med	Unescaped bidder name in Markup	Sanitized with markupsafe.escape(name)
Date Comparison	Logic	High	String comparison rejected 9:30 AM	Parsed with datetime.strptime for comparison
Shill Bidding	Logic	High	Seller bid on own item	Added bidder == self.seller check
Reserve Price	Logic	High	Items won below reserve with Rs.0	Enforced self.highest >= reserve_price
Min Increment	Logic	Med	Accepted bids without +100 increment	Enforced amount >= highest + MIN_INCREMENT
Broad Exceptions	Pylint	Med	W0719 / W0718 bare Exception	Created custom AuctionError & AuctionClosedError
Attribute Outside Init	Pylint	Med	W0201 self.leader defined in method	Refactored leader into dynamic @property
Docstrings & PEP 8	Pylint / Flake8	Low	Missing docstrings, lines > 79 chars	Added PEP 257 docs, wrapped long lines

4. Output Comparison (Before vs After)

Before Fixes: Ben bids 3000: True | Asha bids 3050 (+50): True | Seller bids: True | Bid at 9:30 AM: False | Winner: seller_sam | Final price: 4200.0 | LOT3 winner: carl | HTML/Badge: Raw unescaped XSS
After Fixes: Ben bids 3000: False | Asha bids 3050 (+50): False | Seller bids: False | Bid at 9:30 AM: True | Winner: None (4500 < reserve 5000) | Final price: 0.0 | LOT3 winner: None | HTML/Badge: Fully escaped safe HTML

5. Three Scoped Git Commits

Commit 1 (Bandit Scope): Fix security vulnerabilities reported by Bandit (B411, B701, B704)
Commit 2 (Flake8 Scope): Resolve PEP 8 style, line length, and formatting issues reported by Flake8
Commit 3 (Pylint Scope): Refactor code quality, exception handling, logic bugs, and docstrings for Pylint 10/10

6. Reflection Questions

Q1: Static Analysis in SDLC: Identifies security vulnerabilities, logical flaws, and style discrepancies without code execution early in the development lifecycle ('shift-left').

Q2: Tool Differences & Synergy: Flake8 provides instantaneous style/PEP 8 checks; Pylint performs comprehensive semantic analysis and architectural scoring; Bandit constructs ASTs to identify security risks. Together they cover style, quality, and security.

Q3: Critical Issues Resolved: Neutralizing XSS injection (B701, B704) and XML vulnerability (B411), fixing chronological date parsing, and preventing shill bidding/reserve bypasses restored application safety and integrity.

Q4: Limitations of Static Analysis: Cannot evaluate dynamic runtime state, external network services, concurrency, or domain business logic nuances without dynamic testing.